Distributed Systems

Contents

1 Distributed Systems

A distributed system is a collection of independent and dynamic components that communicate and coordinate with each other, but appear to users as a single coherent system.

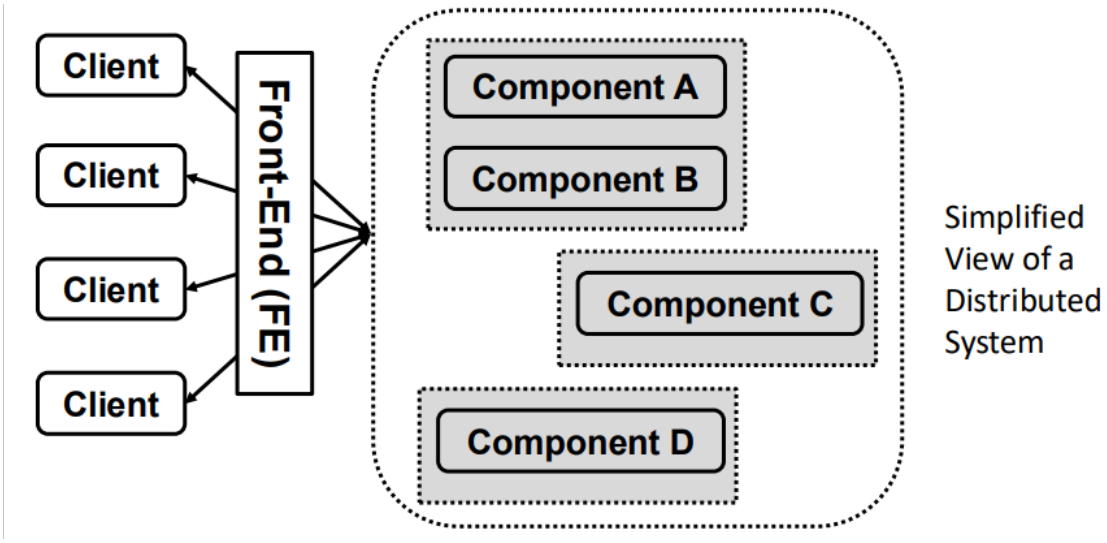
These components are typically distributed across a network and may operate independently, concurrently, and possibly in different physical locations.

Components can include:

- hardware (e.g., servers, sensors)
- software processes
- services (e.g., web services)

Distributed systems enable resource sharing and coordination by allowing components to communicate via message passing over a network.

A key goal is transparency, meaning the system hides the complexity of distribution (such as location, replication, and failures) from users.



The front-end provides system access, preventing direct access to individual components, and supporting transparency.

1.1 Good distributed system design

There are several key principles that define a well-designed distributed (decentralised) system:

- No single node has a complete view of the system state
- Decisions are made using local information
- A single failure should not bring down the entire system
- There is no global clock

Techniques that can be used for this include:

- Asynchronous communication

  - components communicate without waiting for immediate responses, allowing independence and resilience to delays
- Distribution of data and computation
  - data and processing are spread across multiple machines to avoid bottlenecks and single points of failure.
- Caching and replication
  - data is duplicated across nodes to improve availability, reduce latency, and increase fault tolerance.

1.2 Benefits of distributed systems

When deciding between a centralised and distributed architecture, the advantages and disadvantages of each should be evaluated in the context of the application's requirements.

Centralised systems are often more suitable when:

- the system is small-scale
- users and resources are located in a single geographical area
- low latency and simple management are priorities

Distributed systems are generally preferred when scalability, availability, or geographical distribution are required.

The benefits of distributed systems include:

- modularity and faster development
  - systems can be built using reusable components and existing services (e.g. microservices or APIs)
  - this can reduce development time and improve maintainability
- low-cost and scalable
  - multiple inexpensive machines can be used instead of a single high-end machine
  - the system can scale by adding more nodes as demand increases
- improved reliability and availability
  - components can be replicated across multiple machines or locations
  - this reduces the impact of individual machine failures and improves fault tolerance

1.3 Transparency

Transparency is the property of a distributed system that hides:

- the fact that processes and resources are physically distributed across multiple computers (possibly geographically distant and independently managed)
- changes in system components (such as movement, replication, or failure)

The goal is to make the system appear to users as if it were a single coherent system.

A front-end or middleware layer is typically used to achieve transparency, ensuring users interact with the system without needing awareness of its distributed nature.

1.3.1 Transparency requirements

A well-designed distributed system should aim to provide the following forms of transparency:

- Access transparency
  - Hides differences in data representation and access methods
  - Local and remote resources are accessed using the same operations (often via middleware)
- Location transparency
  - Hides the physical location of resources or components
  - Users do not need to know where a resource is stored
- Migration transparency
  - Hides that a resource or component may move from one location to another
  - Example: content distribution networks relocating data between servers
- Relocation transparency
  - Hides that a resource can move while it is being accessed or in use
  - Often supported by techniques like caching or live migration
- Replication transparency
  - Hides that multiple copies of a resource exist
  - Ensures users see a single logical resource while benefiting from faster or more reliable access
- Concurrency transparency
  - Hides that a resource may be shared by multiple users or processes simultaneously
  - The system manages conflicts and consistency automatically
- Failure transparency
  - Hides the failure and recovery of components or resources
  - The system continues to operate despite partial failures where possible

2 Middleware

Middleware is a software layer that enables components in a distributed system to communicate and coordinate with each other while hiding the complexity of distribution.

It provides services and abstractions that allow distributed components to interact as if they were part of a single non-distributed system.

From a user perspective middleware:

- hides implementation details of the system
- hides the distributed nature of components and resources
- provides a unified and transparent interface to the system

From a developer perspective, middleware:

- hides low-level networking and communication details
- provides common programming abstractions and infrastructure for distributed applications
- simplifies communication, coordination, and resource sharing between distributed components

2.1 Benefits of middleware

Developing distributed systems using middleware:

- allows developers to focus on application logic rather than communication mechanisms
- reduces the need to manually implement socket communication and network protocols
- simplifies handling of heterogeneous and dynamically changing network environments

- provides a programming model that makes distributed computing resemble centralised computing

2.2 Types of middleware

Types of middleware include:

- remote procedural call (RPC)
  - a program can invoke a procedure on a remote machine as if it were a local function call
  - middleware handles message passing and network communication automatically
  - commonly used for inter-process communication in distributed systems
- object-oriented middleware (OOM)
  - extends RPC by enabling remote method invocation on distributed objects
  - supports object-oriented programming concepts such as objects, methods, and interfaces
- message-oriented middleware (MOM)
  - enables distributed components to communicate asynchronously through message passing
  - components exchange messages using queues or topics
- web services
  - enable communication between distributed applications over the web
  - uses standard protocols such as HTTP, REST
  - platform-independent
  - commonly used for service-oriented architectures and microservices

2.3 Remote procedural call

Remote Procedure Call (RPC) is a communication technique that allows a program to execute a procedure on a remote machine as if it were a local function call.

RPC hides the complexity of distributed communication, making distributed computing appear similar to centralised computing.

In RPC:

- the system is structured as a collection of procedures/services
- remote function calls are abstracted to appear like local procedure calls
- communication typically follows a request-reply model implemented using message passing
- function parameters and return values are marshalled (encoded into a transferable format) before transmission and unmarshalled after reception

2.3.1 Stubs

RPC uses special helper programs called stubs to hide networking and communication details from programmers.

Client stub The client stub acts as a local representation of the remote procedure.

It:

- receives the client's function call
- marshals the function arguments into a message
- sends the request to the server

- receives the reply message
- unmarshals the returned result
- returns the result to the client program

Server stub The server stub acts as an intermediary between the network and the server procedure.

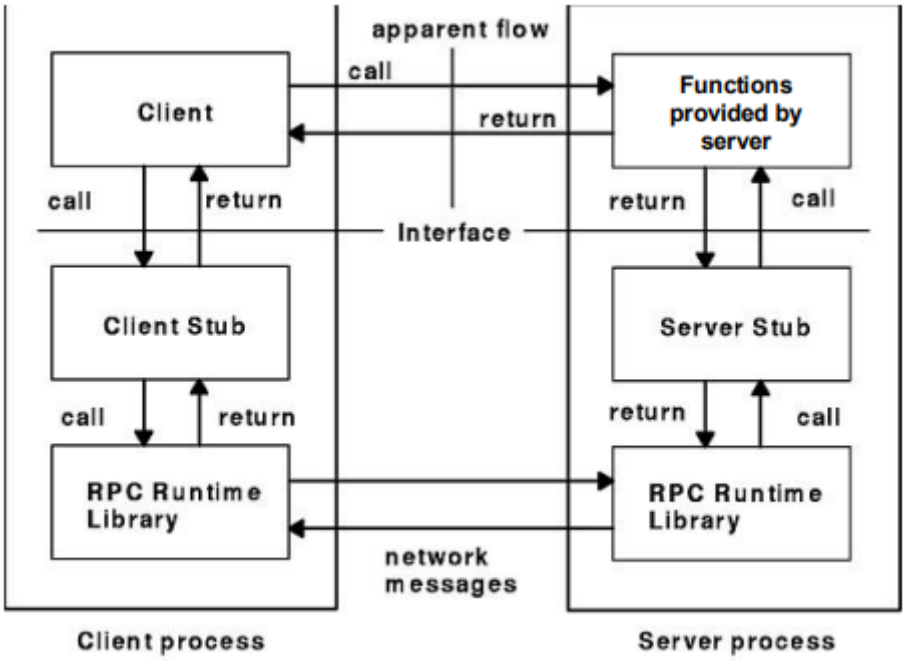
It:

- receives the request message from the client
- unmarshals the arguments
- invokes the actual server procedure
- marshals the procedure's result into a reply message
- sends the reply back to the client

2.3.2 How RPC works

RPC operates as follows:

1. The client calls a procedure as if it were local.
2. The client stub marshals the procedure name and arguments into a request message.
3. The RPC runtime sends the request across the network to the server.
4. The server stub receives the request and unmarshals the arguments.
5. The server executes the requested procedure.
6. The result is marshalled into a reply message.
7. The reply is transmitted back to the client.
8. The client stub unmarshals the result and returns it to the client program.



2.3.3 RPC program development

An RPC system consists of several components involved during development and execution.

Server interface The server defines its service interface using an Interface Definition Language (IDL).

The IDL specifies:

- procedure names
- parameter types
- return types
- callable server operations

Stub compiler The stub compiler:

- reads the IDL specification
- automatically generates:
  - client-side stubs
  - server-side stubs

Application implementation and linking

- the server programmer implements the actual service procedures and links them with the server stubs
- the client programmer implements the client application and links it with the client stubs

RPC runtime library The RPC runtime library manages communication between client and server.

It:

- handles message transmission
- manages connections and communication protocols
- supports marshalling and unmarshalling
- hides low-level networking operations from the application programmer

2.3.4 Properties and limitations

In RPC:

- the request/reply interaction is typically synchronous
  - the client blocks while waiting for a response (or timeout)
  - creates tight coupling between client and server
    - client may be delayed if the server is slow or unavailable
    - server performance can affect client responsiveness
- remote procedures use a call-by-value model
  - arguments are copied and sent to the server
  - no shared global variables between client and server
  - large data structures must be fully transmitted
- location is not fully transparent
  - the client must be able to locate the server (directly or via naming services)
- RPC is procedure-based, not object-based

  - does not natively support object-oriented features like encapsulation or inheritance in the distributed model

2.4 Object-oriented middleware

Object-oriented middleware (OOM) allows objects on one machine to invoke methods on objects located on another machine.

It is similar to RPC, but operates at the level of object methods rather than standalone procedures.

Remote methods can be called as if they were local:

- a client calls a method on a remote object reference (proxy)
- the client-side proxy forwards the method call over the network
- the server receives the request
- a server-side skeleton (stub) invokes the actual method on the real object
- the result is returned to the proxy
- the client receives the result as a normal return value

A remote interface defines which methods of an object can be invoked remotely.

An object request broker (ORB) is responsible for locating and managing remote objects.

2.4.1 OOM application development

To develop an OOM application, the server developer:

- defines the remote interface for each object/service
- implements the methods of the interface
- registers remote objects with the ORB (or naming service/daemon)

2.4.2 OOM application execution

During execution:

- the server:
  - registers objects with the ORB using a name and network location
- the client:
  - looks up a remote object via the ORB (or registry)
  - receives a remote object reference (proxy/stub)
  - invokes methods using normal object syntax
  - interacts with the system as if the object were local

2.4.3 Registry/daemon

A registry (e.g. Java RMI registry) or daemon (e.g. Python service registry) is a background process that maintains mappings between object names and their network locations.

It allows:

- servers to register remote objects
- clients to look up objects by name
- clients to obtain a remote reference (stub/proxy) to invoke methods

2.4.4 Properties of OOM

With object oriented middleware:

- the object-oriented paradigm is supported
  - includes objects, methods, interfaces, encapsulation, and exception handling
  - local objects are typically passed by value (copied)
  - remote objects are passed by reference (via remote object references/proxies)
- the request/reply interaction is typically synchronous (like in RPC)
  - the client blocks while waiting for a response or timeout
- there is location transparency
  - the object request broker maps object names to network locations the physical object location
  - clients do not need to know the physical location of remote objects
- it supports building distributed services across multiple servers
  - objects and services can be located and accessed through the ORB/naming service
  - enables modular, distributed system design using reusable components

2.5 Message oriented middleware

Message-Oriented Middleware (MOM) provides a loosely coupled communication model for distributed systems.

- communication takes place by sending messages
- messages are stored and managed using message queues
- a message broker / message server manages the queues and delivery of messages

In MOM:

- communication is asynchronous and persistent
- client and server are decoupled (communicate indirectly)
- messages are placed into queues managed by middleware (message brokers)
- message servers/brokers handle routing between producers and consumers
- the sender does not need to know the receiver's location or state

2.5.1 Properties of MOM

In MOM:

- communication is asynchronous
  - sender does not wait for receiver to be available
  - improves scalability and loose coupling
  - suitable for distributed application integration
- messages are stored in persistent queues
  - ensures reliability and fault tolerance
  - messages are not lost if receivers are temporarily unavailable
- intermediate message brokers handle processing
  - may perform filtering, transformation, routing, and prioritisation
  - can support logging and monitoring of message flow
  - multiple brokers can form a distributed messaging network
- improves system scalability and flexibility

  - producers and consumers can evolve independently

2.6 Web services

Web services provide a standardised interface over a network (usually the internet) that allows applications to communicate with servers.

- a web service exposes a service interface (API) that defines available operations
- communication is typically done over web protocols such as HTTP/HTTPS
- web services are platform-independent and support machine-to-machine communication
- data is commonly exchanged in structured formats such as JSON or XML

2.7 Client-server interaction (web browser model)

A file or web resource is accessed using a Uniform Resource Locator (URL):

- the URL specifies:
  - the server location (via DNS name)
  - the resource path (file name and directory)
- communication typically uses HTTP or HTTPS
- the browser sends requests and receives responses (usually HTML documents)
- the browser interprets and displays the content (HTML, CSS, JavaScript)

2.7.1 Web browsers

A web browser consists of several components:

- user interface
  - allows users to enter URLs and interact with web pages
- browser engine
  - coordinates actions between components
- rendering engine
  - converts HTML/CSS into the visual layout displayed on screen
- client-sided script interpreter
  - executes JavaScript code embedded in web pages
  - enables dynamic behaviour and interaction
- HTML parser
  - parses HTML into a structured document (DOM tree)
- Networking component
  - handles HTTP/HTTPS requests and responses

2.7.2 Components of web services

Web services use a set of standard technologies to support distributed communication over networks.

Communication

- message content is typically represented in XML (or sometimes JSON)
- SOAP (Simple Object Access Protocol) is used for message-based communication
- a protocol for exchanging structured information between client and server
- supports both synchronous and asynchronous communication

- defines how XML messages are formatted and transmitted over a network (often using HTTP)
- SOAP messages typically include:
  - an envelope (root element containing the message)
  - a header (optional metadata such as authentication or routing info)
  - a body (actual request or response data)

Service description

- WSDL (Web Services Description Language)
  - defines the interface of a web service
  - describes available operations (methods), input/output parameters, and data types
  - specifies how and where the service can be accessed (binding + endpoint URL)
  - acts like a contract between client and server

Service Discovery

- UDDI (Universal Description, Discovery and Integration)
  - a directory service used to locate web services
  - allows services to be published and discovered
  - stores information such as:
    - service name
    - service provider details
    - service URL
    - reference to WSDL description
  - enables clients to search for suitable services dynamically

2.7.3 Attributes of web-services

Web-services:

- use web-based protocols
  - commonly use HTTP/HTTPS for communication
  - web protocols can traverse firewalls and operate across heterogeneous systems and networks
- are interoperable:
  - standard protocols and formats (e.g. SOAP, WSDL, XML) allow systems built with different languages and platforms to communicate
- are XML-based:
  - XML is a standard framework for creating machine-readable documents

2.7.4 REST

Representational State Transfer (REST) is an alternative approach to SOAP-based web services.

- REST is an architectural style for designing distributed web applications
- it treats the web as a collection of resources
- each resource is identified by a unique URL (URI)
- clients interact with resources using standard HTTP methods (e.g., GET, POST)
- RESTful services are typically stateless
  - each request contains all information needed to process it

- REST commonly exchanges data using JSON (though XML may also be used)

2.7.5 JSON

JavaScript Object Notation (JSON) is an open standard, lightweight data-interchange format.

- JSON represents data using attribute-value pairs and arrays
- it uses a human-readable text format
- it is often preferred over XML because it is:
  - simpler
  - less verbose
  - easier to parse

2.7.6 Apache Web Server

The Apache Web Server is a popular and widely used web server software.

It uses a modular architecture:

- functionality is implemented using modules
- each module provides one or more functions/services
- modules interact with the server through hooks
  - a hook is a predefined point in request processing where a module function can be invoked
- for each client request, Apache processes a sequence of hooks
  - different modules can handle different stages of the request
  - e.g. authentication, URL mapping, logging, content generation

2.7.7 Proxy servers

A proxy server acts as an intermediary between clients (e.g. web browsers) and servers.

When a client sends a request:

- the request is first sent to the proxy server
- the proxy forwards the request to the destination server
- the response is returned to the proxy
- the proxy forwards the response back to the client

Proxy servers can be used to:

- cache/store frequently accessed files near users
- reduce network traffic and server load
- improve response time and performance
- provide security, filtering, and access control

Cooperative caching Cooperative caching occurs when multiple proxy servers cooperate to share cached data.

- if one proxy does not contain a requested resource, it may request it from another proxy instead of the original server
- improves response times and reduces overall network traffic
- reduces duplicate downloads across the network

2.7.8 Edge computing

Edge computing is a distributed computing approach in which applications and data processing are performed on servers located closer to end users or devices (at the edge of the network).

Instead of sending all data to distant cloud data centres:

- some computation and storage are performed locally or near the client
- reduces the distance data must travel

Benefits of edge computing include:

- lower latency
  - faster response times for users and real-time applications
- reduced network traffic
  - less data must be transmitted to central cloud servers
- higher effective bandwidth
  - local processing reduces congestion on wide-area networks
- improved reliability and fault tolerance
  - applications may continue operating even if connectivity to the cloud is limited or fails
- improved effectiveness of caching and content delivery
  - web caches and content servers are more efficient when placed closer to clients
- complements cloud computing
  - edge servers handle local/real-time processing while cloud servers provide large-scale storage and computation
  - many cloud providers offer both cloud and edge services

2.7.9 Content Distribution Networks (CDNs)

Content Distribution Networks (CDNs) are large-scale distributed systems used to improve the delivery of web content.

CDNs:

- are networks of edge servers (edge caches) deployed globally as a commercial service
- cache and store copies of web content
  - especially static content such as images, videos, scripts, and webpages -deliver content from the nearest available edge server to the user

When a request is made to a CDN:

- the user requests a resource using a URL
- DNS is used to resolve the domain name
- the CDN's DNS system may redirect the request to a nearby edge server
- if the content is cached at the edge server:
  - it is delivered directly to the user
- if the content is not cached:
  - the edge server fetches it from the origin (main) server
  - the content may then be cached for future requests

3 Replication

Replication is the process of storing copies of the same data or services across multiple nodes or servers in a distributed system.

Replication:

- increases reliability and fault tolerance
  - if one replica fails, another can continue providing the service
  - reduces single points of failure
- improved performance
  - requests can be distributed across multiple replicas (load balancing)
  - reduces load on any single server
- reduces latency
  - users can be served by the nearest replica
  - improves response time, especially in geographically distributed systems
- supports scalability
  - additional replicas can be added to handle increased demand

3.1 Replication considerations

There are several key issues to consider when designing a replication policy in a distributed system:

- Replica placement (location)
  - which nodes or servers should store copies of the data
  - should consider proximity to users, load distribution, and fault tolerance
- Replica quantity
  - how many copies of the data should exist in the system
  - more replicas improve availability and performance but increase cost and maintenance overhead
- Replica consistency
  - how and when replicas are updated to remain consistent
  - determines how up-to-date each copy is across the system
  - involves trade-offs between performance and strict consistency guarantees

These decisions depend on the required performance, availability, and consistency guarantees of the distributed system.

3.2 Replication approaches

There are two main approaches to replication, depending on which layer is responsible for managing replicas:

- Application-level replication
  - the application is responsible for managing multiple copies of data
  - the application must explicitly update all replicas
  - the application must handle consistency and conflict resolution
  - gives more control but increases complexity for developers
- System-level (middleware) replication
  - replication is managed by the operating system or middleware
  - consistency and synchronisation are handled automatically by the system

  - simplifies application development
  - reduces flexibility for application-specific replication strategies

3.3 System components in replication

Different components in a replicated system have different roles:

- Replicas (Rs): back-end servers
  - store and maintain copies of data or services
  - process client requests (read and/or write operations)
  - may be static or dynamic
    - static: fixed number of replicas
    - dynamic: replicas can be added or removed for scalability
- Client (C): request initiator
  - sends read and write requests to the system
  - read requests may be served by a single replica or multiple replicas (for load balancing)
  - write requests may require:
    - propagation of updates to all replicas
    - concurrency control to maintain consistency
- Front-end (FE): mediator / interface layer
  - provides replication transparency to clients
  - hides the existence of multiple replicas
  - routes client requests to appropriate replicas
  - may coordinate responses from multiple replicas
  - monitors replica availability and system state

3.4 System workflow in replication

The system workflow is as follows:

1. An incoming request is recieved by the front-end from the client
2. The front-end forwards the request to one or more replica(s)
3. Replica(s) receive and accept the request
4. Replica(s) determine the ordering of the request relative to other concurrent requests
5. Replica(s) process the request
6. Replica(s) reach a consensus on the effect of the requests
7. One or more replicas send a response to the front-end
8. The front-end may process the response before returning it to the client

3.5 Faults

A server may produce incorrect or inconsistent behaviour due to different types of failures:

- Omission failures
  - a request or response is not received or is lost
  - the server fails to send a response, or the message is dropped in transit
- Commission failures
  - the server responds, but the response is incorrect
  - includes:
    - incorrect processing of requests

        * hardware faults or malfunction
        * software bugs
        * malicious behaviour or attacks

3.5.1 Fault-tolerant service

A fault-tolerant service is a service that continues to provide correct operation despite process or server failures.

Fault-tolerant services typically use replication:

- each replica is assumed to behave according to the system specification when it is not faulty
- if a replica fails, other replicas continue providing the service
- the system masks failures so that clients are not significantly affected

A replicated service is considered correct if it provides:

- Failure transparency
  - the system continues to operate correctly despite failures
  - failures are hidden from the client as far as possible
- Replication transparency
  - clients cannot distinguish between a service implemented with multiple replicas and a single-server implementation
  - the existence of multiple replicas is hidden from the client
  - differences such as location, migration, or replication of data are not visible to users

3.6 Passive replication

In the passive replication (primary-backup) model:

- at any time, there is one primary replica and one or more secondary (slave) replicas
- the front-end communicates only with the primary replica
  - all requests are sent to the primary
  - the primary executes all operations
  - updates are then propagated to backup replicas

3.6.1 Workflow

The workflow in passive replication is:

1. the front-end sends a request (with a unique identifier) to the primary replica
2. the primary replica processes requests sequentially in arrival order
   - it checks the request ID to detect duplicates
   - if the request has already been processed, it resends the stored response
3. the primary executes the request and stores the result
4. if the request is an update operation:
   - the primary sends the updated state (or update log), response, and request ID to all backup replicas
   - backup replicas apply the update and send acknowledgements
5. once acknowledgements are received (depending on the consistency policy), the primary replies to the front-end
6. the front-end forwards the response to the client

3.6.2 Properties of passive replication

Passive replication:

- can tolerate up to f replica failures, provided there are $f + 1$ replicas
  - as long as one replica (the primary) remains operational, the service can continue
- requires relatively little functionality from the front-end
  - the front-end mainly sends requests to the primary
  - if the primary fails, the front-end must locate or switch to a new primary
- has a relatively high communication and update overhead
  - all updates must be propagated from the primary to all backup replicas
  - backups must remain consistent with the primary state

3.7 Active replication

In the active replication model:

- all replicas are state machines with identical roles, organised as a replica group
  - they all start in the same initial state
  - they execute the same operations in the same order
  - therefore, they remain in the same state (deterministic execution assumed)
- if one replica crashes:
  - there is no loss of correctness
  - the service continues using the remaining replicas

3.7.1 Workflow

The workflow in active replication is:

1. the front-end uses reliable totally ordered multicast to send a request (with a unique identifier) to all replicas
   - requests are not handled independently by each replica
2. the multicast ensures that all replicas receive requests in the same total order
3. each replica executes the request independently
   - since all replicas are deterministic state machines and execute in the same order, results are identical
4. no primary or coordination agreement is needed per request
   - consistency is achieved through ordering rather than central control
5. replicas send a response to the front-end
6. the front-end returns one or more responses to the client

3.7.2 Byzantine fault

A Byzantine fault is an arbitrary failure in a distributed system where a component (e.g. a server) behaves incorrectly in any possible way. - the server is operational but produces incorrect, inconsistent, or malicious results - faults may include: - software bugs - hardware faults - network errors - malicious behaviour (e.g. compromised nodes sending false data) - these faults are difficult to detect and handle because behaviour is not simply ‘fail-stop’

Byzantine fault-tolerant algorithms are characterised by a resilience value f, which is the maximum number of faulty processes the system can tolerate.

- to tolerate f Byzantine faulty replicas, the system typically requires:
  - at least $2f + 1$ replicas are needed (maybe $3f + 1$????)
- correct results are determined using majority agreement
- replicas exchange messages and agree on a value despite the presence of faulty or malicious nodes

3.7.3 Properties of active replication

Active replication:

- assumes the existence of a totally ordered, reliable multicast mechanism
  - ensures all replicas receive requests in the same order
- can mask up to f Byzantine failures, if the system incorporates at least $2f + 1$ replicas
  - front end waits until it has collected $f + 1$ identical responses and passes that response back to the client, and discards other responses to the same request
- allows read-only requests to be sent to a single replica
  - this improves performance and reduces overhead
  - however, fault tolerance may be reduced for that specific request
  - if the selected replica fails:
    - the request can be retried on another replica
    - consistency is still maintained because replicas execute operations in the same order

3.8 Passive replication vs active replication

Passive replication:

- provides strong consistency
  - all update requests are processed by a single primary replica
  - primary sends updates to backup replicas to keep them consistent
- provides fault tolerance
  - if the primary fails, a backup replica can take over
- has resource inefficiency
  - backup replicas are mostly idle and not used during normal operation
  - backups are only used when failure occurs
- relatively simple to implement
  - commonly used where strong consistency and correctness are required

Active replication:

- more complex to implement
  - requires totally ordered reliable multicast
  - ensures all replicas receive requests in the same order
- high availability
  - system can continue operating even if some replicas fail
  - all replicas process requests concurrently
- no single point of failure
  - any replica can respond to requests
- potential consistency issues (in practice)
  - relies on correct ordering and deterministic execution
  - failures in ordering/multicast can lead to incorrect or inconsistent behaviour

- used in systems requiring high availability and fast response
  - especially where parallel processing of requests improves performance

4 Cloud computing

Cloud computing is the use of remote servers over a network (typically the internet) to run applications and provide computing resources.

A cloud computing platform provides access to large-scale data centres where computing resources can be leased on demand by users or organisations:

- there is no need to build or manage local physical infrastructure
- resources can be scaled up or down depending on demand
- users pay for what they use

4.1 Cloud computing layers

Cloud computing is commonly structured into four layers:

- hardware
  - physical components such as processors, storage devices, routers, and cooling systems
  - typically hidden from users
- infrastructure
  - provides virtualised computing resources such as virtual machines, virtual storage, and networks
  - uses virtualisation to share physical hardware across multiple users
  - users manage operating systems and applications
- platform
  - provides higher-level services and development platforms
  - includes APIs, databases, and storage services for building applications
- application
  - provides fully developed applications accessed over the internet
  - examples include word processors, spreadsheets, email systems, and presentation tools
  - users do not manage infrastructure or platform components

4.2 Resource allocation

There are two main approaches to resource allocation in distributed and cloud systems:

- Traditional (static allocation)
  - applications run directly on physical servers
  - system administrators manually configure and manage resources
  - resources are fixed and difficult to scale dynamically
  - storage is commonly provided using:
    - Storage Area Networks (SANs)
    - Network Attached Storage (NAS)
- Modern (dynamic allocation)
  - applications run inside virtual machines or containers
  - virtual resources are dynamically mapped onto physical hardware
  - resources can be allocated and scaled automatically based on demand

  - improves flexibility, scalability, and resource utilisation

4.3 Virtualisation

Virtualisation is a technology that creates virtual versions of physical computing resources, such as servers, storage devices, operating systems, or networks.

- virtualisation software abstracts the physical hardware
- this allows multiple virtual machines (VMs) to run on a single physical machine
- each VM behaves like an independent computer with its own operating system and applications

4.3.1 Advantages of virtualisation

Virtualisation has many advantages:

- improved hardware utilisation
  - multiple VMs can share the same physical hardware
  - reduces wasted resources and operational cost
- hardware independence
  - software and operating systems are less dependent on specific physical hardware
  - hardware can be upgraded without significantly affecting applications
- isolation
  - failures, crashes, or attacks in one VM are isolated from others
- ease of portability and migration
  - VMs can be copied, moved, or migrated between physical machines

4.3.2 Mimicking interfaces

Virtualisation works by mimicking or replacing interfaces provided by hardware or software systems.

There are different types of interfaces at different abstraction levels:

Instruction Set Architecture (ISA)

- defines the machine instructions supported by the processor
- includes:
  - privileged instructions
    - can only be executed by the operating system
  - general instructions
    - can be executed by normal applications

System call interface

- provided by the operating system
- allows applications to request OS services such as file access, networking, and memory management

Library/Application Programming Interface (API)

- high-level library calls used by applications
- provides standard interfaces for software development

Depending on which interface is mimicked or replaced, different forms of virtualisation are obtained.

4.3.3 Virtual machines

Virtual machines (VMs) are software-based emulations of physical computers used to virtualise resources such as CPUs, memory, storage, and networking.

- multiple VMs can run on a single physical machine
- each VM has its own operating system and applications

4.3.4 Virtual appliances

A virtual appliance is a pre-configured virtual machine image containing:

- an operating system
- applications and middleware
- predefined configuration settings

Virtual appliances:

- can be deployed quickly without manual installation or configuration
- simplify software distribution and deployment
- provide consistent execution environments across different systems

4.4 Infrastructure-as-a-Service (IaaS)

Infrastructure-as-a-Service (IaaS) is a cloud computing model in which virtualised computing resources are provided over a network.

In IaaS:

- cloud providers rent out virtual machines (VMs) instead of physical machines
- customers can lease resources such as:
  - CPU
  - memory
  - storage
  - networking
- multiple customer VMs may share the same physical hardware
  - virtualisation provides strong isolation between customers
- customers are responsible for:
  - managing the operating system
  - installing applications and middleware
- the cloud provider manages:
  - physical hardware
  - networking
  - storage infrastructure
  - virtualisation layer

IaaS provides:

- flexibility and scalability
- on-demand resource allocation
- reduced cost compared to maintaining physical infrastructure

4.5 Load balancing

Load balancing is a technique used to distribute workload across multiple servers in order to improve resource utilisation and system performance.

- workload (tasks/requests) is distributed across multiple servers
- heavily loaded servers are relieved by shifting work to less loaded or idle servers
- improves overall system efficiency and responsiveness
- prevents bottlenecks and server overload
- load sharing refers to distributing tasks to avoid some servers being idle while others are busy
- load balancing goes further by trying to keep workloads evenly distributed across all servers

4.5.1 Factors in load balancing

Any load balancing algorithm must consider several factors:

- Heterogeneity
  - servers may have different processing capabilities
  - tasks may have different resource requirements
  - load should be assigned according to capacity
- System state information
  - how much information about the system is available
- System objectives
  - load balancing strategies depend on goals such as:
    - minimising response time or delay
    - maximising throughput
    - reducing energy consumption
    - improving availability (e.g., increasing the chance of accepting requests in cloud systems such as IaaS)

4.5.2 Load estimation

For each server, the workload is estimated using a combination of resource utilisation and system indicators:

- Queue length of waiting tasks
  - indicates how many requests are pending execution
  - strongly related to response time and system delay
- CPU / GPU utilisation
- Storage I/O utilisation
- Network bandwidth utilisation
- Application-dependent factors
  - varies depending on the system, such as:
    - number of transactions
    - number of active users
    - media/content access rates
    - request complexity

4.5.3 Load information collection

There are two main approaches for collecting load information:

- Centralised approach
  - a central coordinator collects load information from all servers
  - provides a global view of system state
  - can improve decision accuracy
  - but may become a bottleneck or single point of failure
- Distributed (local) approach
  - each server collects load information from itself and/or neighbouring servers
  - more scalable and fault-tolerant
  - decisions are based on partial or local knowledge of the system state

4.5.4 Task transfer

Task transfer between servers can be classified into two types:

- non-preemptive transfer
  - only tasks that have not yet started execution are transferred
  - the request is moved to another server before execution begins
  - low overhead and simple to implement
  - commonly used for load sharing
  - less effective for fine-grained load balancing
- preemptive transfer
- tasks that are already running can be transferred between servers
- requires saving and transferring the execution state of the task
- expensive due to:
  - state collection (registers, memory, process state)
  - state transmission over the network
  - resumption overhead on the target server
- allows more flexible and fine-grained load balancing

4.5.5 Routing mechanisms

There are two main types of routing in distributed systems, depending on how much of the packet content is used to make routing decisions:

- Layer 4 routing (Transport-layer routing)
  - routes requests based on transport-level information (e.g. IP address and port)
  - does not inspect application content
  - content-blind routing
  - server selection is typically based on:
    - IP header information
    - TCP/UDP connection details (e.g. TCP SYN packets)
  - fast and low overhead
  - limited flexibility in routing decisions
- layer 7 routing (Application-layer routing)
  - examines the application-level content of the request (e.g. HTTP headers, URL, cookies)
  - content-aware routing
  - allows intelligent routing decisions such as:
    - sending different URLs to different servers
    - routing users based on session or data type

  - supports advanced load balancing and service differentiation
  - introduces additional processing overhead and latency

4.5.6 Load distribution algorithms

Common load distribution algorithms include:

- Join-the-Shortest-Queue (JSQ)
  - incoming jobs are assigned to the server with the shortest queue length
  - aims to minimise waiting time
  - simple but requires knowledge of all queue lengths
- Least-Workload (LW) policy
  - assigns jobs to the server with the smallest estimated remaining workload
  - workload is measured as the expected time to finish all currently assigned job
  - more accurate than JSQ because it considers job size, not just queue length
- Round Robin (common baseline algorithm)
  - requests are distributed evenly in a cyclic order across servers
  - does not require knowledge of server load
  - simple and fast, but may be inefficient for uneven workloads
- Random assignment
  - jobs are assigned to servers randomly
  - very low overhead
  - performance depends on chance distribution of load

4.5.7 Static load distribution

In static load distribution:

- the policy is fixed at design time
- scheduling decisions do not depend on the current system state
- decisions are pre-computed or hard-coded into the algorithm
  - e.g. round robin
  - probabilistic/fixed probability assignment
- advantages:
  - simple in implementation
  - low overhead
- disadvantages:
  - cannot adapt to changing workloads
  - may lead to imbalance under dynamic conditions

4.5.8 Dynamic load distribution

In dynamic load distribution:

- the policy is fixed, but decisions are made at runtime
- scheduling decisions depend on the current (or estimated) system state
- uses information such as queue length, CPU load, or response time
- decisions rely on system state information collected via:
  - central coordinator, or
  - distributed/local monitoring

- effectiveness depends on how accurate and up-to-date the system state information is
- advantages:
  - better load balancing under varying workloads
- disadvantages:
  - higher overhead due to state monitoring and communication

4.5.9 Adaptive load distribution

In adaptive load distribution:

- the policy itself can change dynamically at runtime
- it extends dynamic load distribution by adapting both:
  - the decision algorithm
  - the frequency and method of collecting system state information
- the system can adjust based on conditions such as:
  - workload changes
  - server failures
  - network congestion
- advantages:
  - highly flexible and robust under changing conditions
- disadvantages:
  - more complex to design and implement

4.5.10 Components of a load distribution algorithm

A load distribution algorithm consists of several key policies:

- transfer policy
  - determines whether a server should transfer tasks or not
  - defines threshold values based on system metrics such as:
    - number of queued tasks
    - CPU utilisation
    - response time
  - a server becomes a:
    - sender if its load exceeds an upper threshold
    - receiver if its load falls below a lower threshold
  - must consider the overhead and delay of task transfer, since migration is not free
- selection policy
  - determines which tasks should be transferred
  - aims to minimise cost of transfer and improve performance
  - selection criteria may include:
    - estimated execution time
    - task size or resource requirements
    - expected impact on server response time
- location policy
  - determines which server will receive the transferred task
  - may use:
    - probing/polling to find lightly loaded servers
    - random selection or heuristics

    - can be implemented:
        * sequentially (one server at a time)
        * in parallel (e.g., multicast to multiple servers)
- information policy
    - determines when, where, and what load information is collected and shared
    - influences accuracy and overhead of the algorithm
    - can be:
        * demand driven: load information is collected only when a transfer decision is needed
        * periodic: servers exchange load information at fixed time intervals
        * state-change-driven: load information is shared when a server's state changes significantly

4.5.11 Initiation of load distribution

Load distribution can be initiated in two main ways:

- sender-initiated
    - an overloaded server initiates load balancing
    - triggered when load exceeds an upper threshold
    - the sender tries to find a suitable receiver (lightly loaded server)
    - advantages:
        * simple implementation
        * effective under light to moderate system load
        * newly arriving tasks can be transferred before execution (low overhead)
            · no need for process state transfer in non-preemptive transfer
    - disadvantages:
        * under high system load, most servers are also busy
            · difficult to find available receivers
        * excessive probing/polling can degrade performance
        * may lead to unstable behaviour when many senders compete
- receiver-initiated
    - an underloaded server initiates load balancing
    - triggered when load falls below a lower threshold
    - the receiver tries to find a sender with excess load
    - advantages:
        * performs well when the system is heavily loaded
        * helps utilise idle capacity more effectively
        * tends to be more stable under high load conditions
    - disadvantages:
        * under high load, it may be difficult to find lightly loaded senders
        * frequent polling/searching introduces overhead
        * may require task migration, which can be expensive (especially if preemptive transfer is needed)
        * less effective when most servers are already overloaded

Symmetric initiation In symmetric initiation:

- both senders and receivers can initiate load balancing
- at low system load:

  - senders can easily find receivers
  - system has enough idle capacity to accept transferred tasks
- at high system load:
  - receivers can easily find senders
  - overloaded servers are more common, so work is easier to locate and transfer
- disadvantages:
  - combines disadvantages of both sender- and receiver-initiated approaches
  - can lead to high overhead due to polling/search activity at high load
  - receiver-initiated transfers may involve preemptive task migration, which is expensive
  - system can become unstable if too many nodes simultaneously initiate load balancing
- can be optimised by searching local neighbouring servers first#
  - improves efficiency by reducing network-wide probing
  - helps balance between responsiveness and communication cost

Adaptive initiation In adaptive initiation:

- polling is limited and adaptive rather than continuous or global
- each server maintains a local view of system state using three lists:
  - sender list (overloaded servers)
  - receiver list (underloaded servers)
  - OK list (balanced/normal servers)
- initially, each server is assumed to be a receiver (not overloaded)

Sender behaviour:

- a sender selects a potential receiver from the head of its receiver list
- it polls the selected server
- the polled server:
  - moves the sender to the head of its sender list
  - responds with its current state
- if the polled server is a receiver:
  - the task is transferred immediately
- otherwise:
  - the sender updates its lists accordingly
  - and continues polling the next candidate
- if no receiver is found, the task may still be transferred via receiver-initiated interaction

Receiver behaviour:

- a receiver searches for potential senders to obtain work
- servers are queried in the following order:
  - sender list (head to tail)
  - OK list (tail to head)
  - receiver list (tail to head)
- if a suitable sender is found, a task is transferred from that sender

Properties of adaptive initiation In adaptive initiation:

- polling overhead is reduced compared to static approaches
- system dynamically adjusts behaviour based on load conditions
- at high load:

  - most servers become senders
  - receiver-initiated behaviour dominates
  - sender polling decreases as fewer receivers exist
- at low load:
  - most servers are receivers
  - sender-initiated transfers dominate
  - senders can easily find receivers
- system automatically adapts between:
  - sender-initiated behaviour (low load)
  - receiver-initiated behaviour (high load)
- improves stability and efficiency under varying load conditions
- allows non-preemptive transfers when possible, reducing overhead

4.5.12 Selecting a load distribution algorithms

Different load distribution algorithms are useful for different applications:

- If the system is rarely heavily loaded:
  - sender-initiated algorithms work best
  - senders can easily find idle receivers
  - low overhead and efficient task transfer
- If the system is frequently heavily loaded:
  - receiver-initiated algorithms work best
  - many overloaded servers exist, so receivers can find senders easily
  - better utilisation under high load
- If load fluctuates widely
  - symmetric algorithms are preferred
  - both sender- and receiver-initiated approaches are used
  - provides better balance across changing conditions
- If load fluctuates widely and preemptive migration is expensive
  - sender-initiated algorithms are preferred
  - avoids costly state transfer of partially executed tasks
  - uses non-preemptive transfers where possible
- If workload is highly heterogeneous (unpredictable)
  - adaptive algorithms are preferred
  - system dynamically adjusts between sender and receiver behaviour
  - provides best robustness under changing conditions

4.5.13 Power-of-d load balancing

When there are many servers, the Power-of-d policy improves load distribution with low overhead:

- for each incoming job, randomly select d servers
- assign the job to the server with the smallest queue length among those d servers
- reduces the chance of sending work to heavily loaded servers compared to purely random assignment
- achieves near-optimal load balancing with minimal information overhead

4.5.14 Join-the-idle-Queue (JIQ)

In the Join-the-idle-Queue (JIQ) policy:

- servers notify the load balancer when they become idle
- the load balancer maintains a list of idle servers
- when a job arrives:
  - if idle servers exist:
    - assign the job to one of the idle servers from the list
  - if no idle server exists:
    - assign the job to a randomly selected server
- once a server becomes busy again:
  - it is removed from the idle list

5 Concurrent events and consistency control

In distributed systems, client transactions often access or update objects stored across multiple servers:

- each transaction consists of a sequence of operations
- operations may involve reading or writing shared data on one or more servers

Sometimes transactions interfere with each other:

- multiple transactions may access the same object concurrently
- this can lead to conflicts such as inconsistent or incorrect results

A shared object is an object that can be accessed simultaneously by multiple transactions or events.

- servers managing shared objects must ensure correctness under concurrent access
- this is done through concurrency control
- concurrency control ensures that concurrent executions do not lead to inconsistent states
- a common form of concurrency control is locking

5.1 Locking and timestamp ordering

Locking and timestamp ordering are covered in databases.

5.2 Network Time Protocol (NTP)

The Network Time Protocol (NTP) is a protocol used to synchronise the clocks of computers and devices over a network:

- NTP synchronises clocks using a hierarchy of time servers, organised into stratum levels
- lower stratum levels are closer to highly accurate reference clocks

5.2.1 Stratum levels

- Stratum 0
  - highly accurate reference clock
  - e.g, atomic clocks, GPS clocks
- Stratum 1
  - directly connected to stratum 0 devices

  - act as primary time servers
  - usually operated by research institutions or councils, and major ISPs
- Stratum 2, 3, …
  - obtain time from higher-level servers
  - propagate accurate time throughout the network

5.3 Consistency

If a distributed system maintains only a single copy of each object:

- correctness depends only on the sequence of operations applied to that object
- all clients observe the same data state

Maintaining consistency becomes more difficult when objects are replicated across multiple machines:

- different replicas may receive operations in different orders
- this can occur because of:
  - network latency
  - clock synchronisation errors
  - message loss or retransmission
  - concurrent updates
- as a result:
  - replicas may temporarily store different values for the same object

5.3.1 Consistency model

A consistency model is a contract between a distributed data store and processes (clients):

- it defines the behaviour and results of read and write operations in the presence of concurrency and replication
- it specifies what value a read operation is allowed to return

Most consistency models aim for reads to return the result of the 'latest' write operation.

- different models define 'latest' differently
- stronger consistency models enforce stricter ordering rules
- weaker consistency models relax ordering requirements to improve:
  - performance
  - scalability
  - availability

5.3.2 Linearizability

Linearizability is a strong consistency model in distributed systems.

It requires that operations appear as if they were executed:

- on a single correct copy of the data
- in some sequential (interleaved) order
- while remaining consistent with real-time ordering

Virtual interleaving Linearizability assumes there exists a virtual canonical execution:

- operations from multiple clients are interleaved into one global sequence
- this sequence behaves as if all operations occurred on a single machine

Each client observes a view of the shared objects that is consistent with this single global order.

Real-time ordering requirement If operation x completes before operation y begins in real time:

- then x must appear before y in the virtual sequence

Formally:

- if $TS(x) < TS(y)$
- then $OP(x)$ must precede $OP(y)$

where:

- TS = timestamp / real-time ordering
- OP = read or write operation

5.3.3 Sequential consistency

Sequential consistency is a weaker consistency model than linearizability.

A system is sequentially consistent if:

- there exists some interleaving of all operations from all processes
- such that:
  - the interleaved sequence produces results consistent with a single correct copy of the data
    - i.e., read/write operations behave as if executed on one machine
  - for each individual process, the order of operations in the interleaving must match the order in the program
    - i.e., each process sees its own operations in the correct sequence

All processes agree on a single global order of operations but this order does not need to respect real-time ordering.

Sequential consistency vs linearizability

- sequential consistency:
  - ignores real-time constraints
  - only preserves program order per process
- linearizability:
  - preserves real-time ordering between operations across processes

5.3.4 Casual consistency

Causal consistency is a weaker form of sequential consistency that distinguishes between operations that are causally related and those that are not:

- If one operation causally influences another, all processes must see them in the same order
- If operations are independent (concurrent), different processes may see them in different orders

If event B is caused or influenced by event A, then:

- all processes must observe A before B

Concurrent operations Operations are concurrent if they are not causally related

- no dependency exists between them
- different replicas may apply them in different orders

5.3.5 Eventual consistency

Eventual consistency is a very weak consistency model used in large-scale distributed systems:

- replicas are allowed to be temporarily inconsistent
- but all replicas eventually converge to the same value

6 Fault tolerance

Fault tolerance is the ability of a system to continue correct operation even when faults occur.

- faults may include:
  - hardware failures
  - software bugs
  - network errors
  - process crashes
- a fault-tolerant system aims to mask or recover from failures so users still observe correct behaviour

6.1 Measuring system quality

Two common measures of distributed system quality are:

- reliability
  - the ability of a system to operate without failure over time.
  - measures how long a system runs before failing
  - describes the probability of correct operation over a time interval
  - often expressed using MTBF (Mean Time Between Failures)
    - average time between one failure and the next
- availability
  - the proportion of time a system is operational and accessible.
  - measures how often the system is usable
  - includes downtime due to:
    - failures
    - repairs
    - maintenance

$$A = \frac{MTBF}{MTBF + MTTR}$$

where

- MTBF = Mean Time Between Failures
- MTTR = Mean Time To Repair

6.2 Redundancy

Redundancy improves fault tolerance by replicating system components or operations, so that failures can be masked or recovered from.

6.2.1 Time redundancy

Time redundancy involves repeating an operation over time to improve reliability.

- the same action is executed more than once if a fault occurs or is suspected
- relies on the idea that a correct execution will eventually succeed

It is suited for:

- transient faults (occur once and disappear)
  - e.g., packet loss
- intermittent faults (appear and disappear repeatedly)
  - e.g., condition dependant bugs

Properties

- the same operation is performed multiple times
- if results are consistent across repetitions, correct execution is assumed
- helps detect and overcome temporary faults
- does not handle permanent faults effectively

Advantages

- simple to implement
- effective against temporary and intermittent failures
- no need for extra hardware

Disadvantages

- increases execution time (performance overhead)
- cannot fix permanent hardware/software faults
- may waste resources by repeating correct operations

6.2.2 Component redundancy

Component redundancy improves fault tolerance by replicating system components and using their outputs to detect or mask faults.

- multiple independent components are created
- each component performs the same functionality
- outputs are compared to detect inconsistencies
- a correct result is selected (e.g. majority voting or agreement)

Properties

- provides strong fault detection and tolerance
- can mask failures if at least one component is correct
- typically has low performance impact compared to time redundancy

- computations happen in parallel rather than repeated over time

N-Version Programming (NVP) N-Version Programming is a form of component redundancy using design diversity:

- multiple independent versions of the same program are developed
- each version is implemented by different teams or methods
- all versions receive the same input and execute in parallel
- outputs are compared using a voter mechanism

NVP:

- can tolerate:
  - hardware faults
  - some software faults (implementation errors in one version)
- relies on assumption of independence between versions

Advantages

- high reliability
- parallel execution reduces delay compared to time redundancy
- effective against random independent faults

Disadvantages

- expensive to design and develop (multiple versions required)
- difficult to ensure true independence between versions
- vulnerable to common-mode failures (correlated faults)

6.2.3 Information redundancy

Information redundancy improves fault tolerance by adding extra bits to data so that errors can be detected or corrected.

- extra (redundant) information is encoded into data before storage or transmission
- receiver uses this extra information to:
  - detect errors
  - and sometimes correct them
- typically implemented using error-detecting or error-correcting codes

Information redundancy techniques

- Parity check
  - data is divided into fixed-size bit groups (words)
  - an extra parity bit is added based on XOR of bits
  - used to detect single-bit errors
- Checksum
  - a fixed-size value computed from a block of data
  - sent along with the data
  - receiver recomputes checksum and compares values
  - used to detect corruption during transmission or storage

Design principle

- redundancy is chosen to:
  - minimise extra overhead
  - while still being effective for expected fault types
- stronger error correction requires more redundancy

Advantages

- less hardware required compared to component replication
- effective for error detection in communication and storage systems
- lightweight compared to full system redundancy

Disadvantages

- added complexity in encoding and decoding
- limited ability to recover from errors (often only detects, not corrects)
- ineffective against large or systematic faults

6.2.4 Communication redundancy

Communication redundancy improves fault tolerance by repeating, recovering, or rerouting messages in distributed communication to handle message loss or server failures.

Common communication problems include:

- client cannot locate server
- client request is lost in the network
- server crashes after receiving request
- server response is lost before reaching client

Handling communication failures If the client cannot locate server / request is lost:

- use an exception handler (language-dependent)
- query a directory service to find updated server locations
- apply a timeout mechanism
  - if no response, retry request
- repeated failures may indicate:
  - server is down or unreachable
- retrying is safe mainly for idempotent operations
  - repeating the operation has no additional effect

If the server crashes after receiving request:

- server may fail:
  - before executing request
  - after executing request but before replying
- solutions:
  - store request at front-end or middleware
  - use backup/alternative server
  - rebuild state and retry request
  - or report failure depending on delivery guarantees

If the server reply is lost:

- request was processed, but response did not reach client
- solutions:
  - client uses timeout and retry mechanism
  - server may need to detect duplicate requests
  - system should avoid re-executing non-idempotent operations incorrectly

6.2.5 General workflow towards fault tolerance

- Error detection & diagnosis
  - detect that a fault has occurred and identify its source
- Failure isolation
  - isolate the faulty component from the rest of the system
  - prevent it from affecting other components
- Error containment
  - confine the effects of the fault to a limited area
  - prevent error propagation through the system
- Recovery
  - restore the system to a correct state
  - using rollback, restart, or replacement of components

6.3 Error recovery

Error recovery is used to restore a distributed system to correct operation after a failure. There are two main approaches:

- backward recovery
  - return the system to a previous correct (failure-free) state
  - execution resumes from a saved checkpoint
- forward recovery
  - move the system to a new correct state after a failure
  - avoids rollback by correcting the error and continuing execution

6.3.1 Backward recovery

Backward recovery is commonly implemented using checkpointing:

- each process periodically saves its state (checkpoint)
- checkpoints are stored so execution can be restarted after failure
- a global checkpoint is formed from a consistent set of local checkpoints
- recovery restarts from the most recent consistent global checkpoint
  - this is called the recovery line

Types of checkpointing

- Uncoordinated checkpointing
  - each process checkpoints independently
  - simple and flexible
  - may cause useless checkpoints and the domino effect
    - domino effect is a cascading rollback where one process failure forces other processes

to repeatedly roll back to earlier and earlier checkpoints due to inconsistent global state

- Coordinated checkpointing
  - all processes coordinate to take a consistent global checkpoint
  - avoids domino effect
  - higher overhead since processes may need to pause
- Communication-induced checkpointing
  - processes take checkpoints based on information in exchanged messages
  - combines local and forced checkpoints
  - avoids domino effect without full global synchronisation
- Message-Logging
  - infrequent checkpoints are taken
  - all messages are logged between checkpoints
  - recovery replays logged messages to rebuild state
  - reduces rollback distance but increases logging overhead

6.3.2 Issues of backwards recovery

- restarting the system from a recovery line may require re-executing previously completed operations
  - leads to wasted computation and reduced efficiency
- checkpoints may not always be sufficiently recent or consistent
  - in worst cases, recovery may roll back to the initial state
  - this is caused by the domino effect, where repeated rollbacks propagate backwards through multiple checkpoints

6.3.3 Forward recovery

Forward recovery is an approach where the system continues operation without rolling back, by moving into a new correct state after a fault occurs.

It focuses on avoiding system interruption by correcting or masking errors at runtime.

6.3.4 Method used in forward recovery

- Self-checking components
  - components detect their own faults during execution
  - faulty components can be isolated or replaced
- Component switching
  - when a failure is detected, the system switches to a non-faulty replica
  - both components perform the same task code
- Fault masking
  - errors are hidden using redundancy mechanisms
  - example:
    - voting schemes used to mask Byzantine faults
    - majority result is accepted even if some components are faulty
- Error compensation / resilience
  - system continuously compensates for errors using redundant computation
  - based on diversity and parallel execution of processes

    - examples:
        * multiple processes run in parallel and combine results
- Data predication:
    - system estimates missing or delayed data to maintain continuity
    - used when real-time data is temporarily unavailable
    - example:
        * online gaming predicts player movement when packets are lost

6.3.5 Forward recovery vs backward recovery

Backward recovery

- requires no knowledge about the specific error
- restores system to a previously saved error-free state
- relies on checkpoints (state saving)
- application-independent
- limitations:
    - time and storage overhead for maintaining checkpoints
    - possible large rollback (domino effect)
    - may repeat previously executed work

Forward recovery

- requires knowledge of the error to apply a correction strategy
- moves system to a new correct state without rollback
- does not rely on restoring old checkpoints
- application-dependent
- advantages:
    - more time and space efficient
    - avoids re-executing past computations
- used when system delays or interruption are unacceptable